

Song Recommendations using Semantic IDs and LLM-Driven Query Handling

Motivations

Music streaming platforms like Spotify and Apple Music have become vital platforms for people discovering and engaging with music. These platforms allow people to find new artists, connect with people with similar music tastes, and share playlists or songs with friends and family. Along with this, music is an integral part of many people's lives, with Spotify having over 290 million premium subscribers (Singh et al) and Apple Music having over 94 million subscribers (Kumar et al). However, despite having millions of monthly subscribers, the song recommendation systems that power these platforms still struggle to move users beyond what is already popular or heavily promoted. Many users develop deeply personal listening habits over time, forming a rich and detailed signal of their music taste that current recommendation systems do not fully exploit.

Our goal for this project is to create a song recommendation system based on a user's listening history in order to recommend songs that the user may not have considered listening to, or even heard of in the first place. We plan to utilize listening history data, such as artist preferences, genre patterns, and listening frequency in order to showcase songs that are relevant to the user's music preferences instead of just pushing whatever the most popular or mainstream songs are. For example, there are many instances where a user may frequently listen to a certain artist or genre, but similar artists who produce the same style or genre of music might go unnoticed. Our goal is to give that user more options and opportunities to explore music beyond their immediate awareness. Beyond just providing value to the user, this song recommendation system can help improve artist visibility and engagement on streaming platforms.

Problem Statement

Song discovery on streaming platforms relies heavily on popularity signals and broad genre filters to recommend music, and the personalized recommendation systems that exist today offer little connection between a user's deeper listening habits and truly novel music discovery. This leads to many users of music streaming platforms, regardless of how long they've been a user, to be repeatedly recommended the same popular or already familiar songs that might not fully align with their personal music taste. This not

only limits what opportunities the user is presented with, but also how much exposure smaller and less popular artists and songs receive on these platforms.

Our group aims to solve this problem by creating a recommendation system based on a user's listening history to recommend songs that are relevant to the user's listening habits and music taste.

Justification

Many recommendation systems rely on popularity-based ranking, which prioritizes items with highest global interactions and takes into consideration the tastes of all users. Though this makes it simple and scalable, it introduces strong bias and limits exposure of songs that may be more niche or less popular, heavily reducing diversity of the recommendations.

In particular, collaborative filtering is good for this problem because it is specifically user and item based, which makes it perfect for improving personalization of a recommendation system. Numerous techniques such as matrix factorization show a certain effectiveness in capturing any developing relationships between users and items. However such techniques still face limitations. They struggle with sparse data within a set of large datasets, as users will typically interact with only a small subset of available songs, and they also tend to reinforce already existing listening song patterns rather than the encouraging exploration of new songs, which isn't ideal. This results in something called a filter bubble.

More recently there has been work that explores more hybrid methods that combine collaborative filtering with content based methods. By combining metadata such as genre and lyrics, these systems can better capture item similarity and improve recommendations for less popular songs while also maintaining a certain degree of personalization. Even so, many existing implementations either rely too heavily on one or the other and still result in some forms of bias. Our approach builds upon these pre-existing foundations by combining item based collaborative filtering with content based similarity that is derived from song metadata. Our model will emphasize upon recommending lesser, more unknown songs as opposed to popular ones based on a user's listening history and preferences. Additionally, we will integrate metadata features such as genre, artist familiarity, lyrical context, and other additional data to help our system mitigate sparsity and improve the recommendations of not so popular songs.

The novel aspects of our project are that we will use LLMs to modify and enhance our recommendations. Spotify Research published research at RecSys that uses LLMs to turn ambiguous and general queries into better routing for recommendation systems (Palumbo et al) . This may help our project by improving

the translation between what a user might ask, such as “study music” into the latent categories that we use for our model. Also, content-based features (“semantic IDs”) can improve the accuracy and diversity in music recommendation by enriching our item representations (Mei et al). We could build upon our approach’s novelty by adding metadata, artist, and audio features to our MSD-based CF model. Finally, we follow recent trends in diversity-aware recommendations by explicitly encouraging diversity across artists and audio features in our top-K recommendations (Zhao et al).

Methodology and Approach

Our group will use item-based collaborative filtering based upon interaction data. This means that we will first prepare the user song playing histories into a user-item matrix. We will also remove very rare users and songs to limit the dimensions of our data and reduce some sparsity since the distribution of listens per song or user will be fairly long-tailed. This step will also involve some data cleaning with renaming columns, removing missing values, or filtering our users with minimal listens. Then, we will build a baseline recommender that uses popularity to recommend the most played songs overall. This will involve grouping listens by a unique song identifier, the sum of play counts, and then recommending the overall most listened songs as our naive recommendation. We will create the user-item matrix from the interaction table. Rows will be each user, and the columns will be each song (so the cell values will then become rows). We will train collaborative filtering using a user-based system with matrix factorizations such as ALS. This involves computing similarity between user vectors, and then finding the most similar users for each target user. Then we aggregate the songs those neighbors listened to, and then rank songs by weighted similarity score. Good libraries for this include sklearn’s cosine similarity metrics and surprise (a Python library for recommender systems).

As an extension, we can add some additional content-based features to improve our behavior. We can use song metadata such as artist, genre, tags, and lyrics and then compute similarity using TF-IDF or embeddings. Some good Python libraries for this include (TfidfVectorizer, cosine_similarity). Another extension we are considering is motivated by recent RecSys 2025 work on LLM-based query understanding. We will build a lightweight LLM-query-refinement step that maps natural-language user intents (e.g., “study music”) to feature-space concepts that our existing recommendation model can use. This is not a full end-to-end LLM recommender, but a bridge between user language and our CF-plus-metadata pipeline.

Dataset description and data acquisition methods

Because the dataset comes from a closed Kaggle competition, the files attached to the Kaggle competition page can no longer be downloaded directly. Therefore, some of these files were downloaded from public GitHub mirrors. The download link in the appendix section.

train_triplets.txt contains 48,373,586 user-song interactions, with each line representing one interaction. The file has three columns: `user_id`, `song_id`, and `play_count`, which represent the user, the song, and the number of times the user interacted with that song.

kaggle_visible_evaluation_triplets.txt has the same format as `train_triplets.txt`, this file contains the visible half of the evaluation users' listening histories, with a total of 1,450,933 records.

year1_test_triplets_hidden.txt and **year1_valid_triplets_hidden.txt** are the hidden test/valid files released by the host.

kaggle_songs.txt is the canonical list of 386,213 songs, containing all song IDs from the training and test data, including both the visible and hidden parts.

kaggle_users.txt is the canonical list of 110000 user identifiers, which is used to sort the predictions with respect to the submission file.

taste_profile_song_to_tracks.txt contains 386,213 entries and is included in the original Kaggle MSD Challenge data files. It maps Taste Profile song IDs to Million Song Dataset track IDs.

track_metadata.db contains metadata for exactly 1,000,000 songs. The dataset contains many metadata fields, but for simplicity, we only introduce the fields that we plan to use, as shown in the following table. (For any numerical values, 0 means missing)

A detailed description of each column can be found in our appendix. Our approach to handling these missing values is to use mean imputation followed by mean-centering. After this process, the imputed missing values will be centered to zero.

Evaluation method and plan

We will use the same evaluation method as the Million Song Dataset Challenge, which uses mean average precision truncated at 500 (MAP@500). For each user, we will recommend 500 songs and evaluate the model based on how many of those songs actually appear in the user's listening history in the validation set. We will compare our model's performance with the performance of the past winning model. Hopefully, we can achieve better results by adding novelty to our project.

Citations

Singh, Shubham. “Spotify Statistics (2024) – User Growth, Top Artists & More.” *DemandSage*, 15 Jan. 2025, www.demandsage.com/spotify-stats/.

Kumar, Naveen. “Apple Music User Statistics 2026 (Revenue & Market Share).” *DemandSage*, 31 December 2025, www.demandsage.com/apple-music-statistics/.

Mansoury 22, Juno Prent 11 Masoud, et al. *Correcting for Popularity Bias in Recommender Systems via Item Loss Equalization*, arxiv.org/html/2410.04830v1.

Liu, Zhenghao, et al. “Text Matching Improves Sequential Recommendation by Reducing Popularity Biases.” *arXiv.Org*, 27 Aug. 2023, arxiv.org/abs/2308.14029.

Chen, Jiawei, et al. “Bias and Debias in Recommender System: A survey and future directions.” *ACM Transactions on Information Systems*, vol. 41, no. 3, 7 Feb. 2023, pp. 1–39, <https://doi.org/10.1145/3564284>.

Darius Afchar, Romain Hennequin, and Vincent Guigue. 2023. “Of Spiky SVDs and Music Recommendation.” In *Seventeenth ACM Conference on Recommender Systems (RecSys '23)*, September 18–22, 2023, Singapore, Singapore. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3604915.3608850>

J. Mei, F. Henkel, S. E. Sandberg, O. Bembom, and A. F. Ehmman, “Semantic IDs for Music Recommendation,” *Proceedings of the 19th ACM Conference on Recommender Systems (RecSys '25)*, 2025.

Available at: <https://arxiv.org/abs/2507.18800> and <https://doi.org/10.1145/3705328.3748139>.

Y. Zhao, Y. Wang, Y. Liu, X. Cheng, C. C. Aggarwal, and T. Derr, “Fairness and Diversity in Recommender Systems: A Survey,” *ACM Transactions on Intelligent Systems and Technology*, vol. 16, no. 1, article 2, pp. 1–28, 2025.

Available at: <https://doi.org/10.1145/3664928>.

E. Palumbo, M. Isaksson, A. Tamborrino, M. Movin, C. Dincu, A. Vardasbi, L. Nikeshkin, O. Gorobets, A. Nyman, P. Newdick, et al., “You Say Search, I Say Recs: A Scalable Agentic Approach to Query Understanding and Exploratory Search at Spotify,” Proceedings of the 19th ACM Conference on Recommender Systems (RecSys '25), 2025.

Available at: <https://doi.org/10.1145/3705328.3748127>.

Appendix

Table Description

	Meaning	Completeness	Data type
track_id	Identifies a specific track/version of a song. One song_id can map to multiple track_ids.	100%	Text
title	Song title.	100%	Text
song_id	Song identifier used to represent the song.	100%	Text
release	Album or release name.	100%	Text
artist_id	Echo Nest artist ID.	100%	Text
artist_name	Artist name.	100%	Text
duration	Track duration in seconds.	100%	Numerical
artist_familiarity	Echo Nest estimate of how familiar or well-known the artist is.	99.75%	Numerical
artist_hotness	Popularity-buzz score for the artist.	94.95%	Numerical
year	Release year.	51.56%	Numerical

Dataset download links

http://millionsongdataset.com/sites/default/files/challenge/train_triplets.txt.zip

https://raw.githubusercontent.com/himanshugupta1009/Music-Recommendation-System/master/kaggle_visible_evaluation_triplets.txt

https://raw.githubusercontent.com/djain-cbs/Million-song-dataset-recommendar-system/master/kaggle_songs.txt

https://raw.githubusercontent.com/ran784388220/million-songs-dataset/master/kaggle_users.txt

https://raw.githubusercontent.com/sastafford/million-song-dataset/master/data/taste_profile_song_to_tracks.txt

http://millionsongdataset.com/sites/default/files/AdditionalFiles/track_metadata.db

<http://millionsongdataset.com/sites/default/files/challenge/EvalDataYear1MSDWebsite.zip>